torch.linalg.pinv#

https://pytorch.org/docs/stable/generated/torch.linalg.pinv.html

Description:

Computes the pseudoinverse (Moore-Penrose inverse) of a matrix. The pseudoinverse may be defined algebraically but it is more computationally convenient to understand it through the SVD Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if A is a batch of matrices then the output has the same batch dimensions. If hermitian= True, A is assumed to be Hermitian if complex or symmetric if real, but this is not checked internally. Instead, just the lower triangular part of the matrix is used in the computations. The singular values (or the norm of the eigenvalues when hermitian= True) that are below max(atol,σ1·rtol)\max(\text{atol}, \sigma_1 \cdot \text{rtol})max(atol,σ1·rtol) threshold are treated as zero and discarded in the computation, where σ1\sigma_1σ1 is the largest singular value (or eigenvalue).

Function Signature

```
torch.linalg.pinv(A, *, atol=None, rtol=None,
hermitian=False, out=None) → Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

atol (float, Tensor, optional) – the absolute tolerance value. When None it’s considered to be zero. Default: None. rtol (float, Tensor, optional) – the relative tolerance value. See above for the value it takes when None. Default: None. hermitian (bool, optional) – indicates whether A is Hermitian if complex or symmetric if real. Default: False. out (Tensor, optional) – output tensor. Ignored if None. Default: None.

Code Examples

```
torch.linalg.lstsq(A, B).solution == A.pinv() @ B
```

```
>>> A = torch.randn(3, 5)
>>> A
tensor([[ 0.5495,  0.0979, -1.4092, -0.1128,  0.4132],
        [-1.1143, -0.3662,  0.3042,  1.6374, -0.9294],
        [-0.3269, -0.5745, -0.0382, -0.5922, -0.6759]])
>>> torch.linalg.pinv(A)
tensor([[ 0.0600, -0.1933, -0.2090],
        [-0.0903, -0.0817, -0.4752],
        [-0.7124, -0.1631, -0.2272],
        [ 0.1356,  0.3933, -0.5023],
        [-0.0308, -0.1725, -0.5216]])

>>> A = torch.randn(2, 6, 3)
>>> Apinv = torch.linalg.pinv(A)
>>> torch.dist(Apinv @ A, torch.eye(3))
tensor(8.5633e-07)

>>> A = torch.randn(3, 3, dtype=torch.complex64)
>>> A = A + A.T.conj()  # creates a Hermitian matrix
>>> Apinv = torch.linalg.pinv(A, hermitian=True)
>>> torch.dist(Apinv @ A, torch.eye(3))
tensor(1.0830e-06)
```

Notes & Warnings

NOTE Note This function uses torch.linalg.svd() if hermitian= False and torch.linalg.eigh() if hermitian= True. For CUDA inputs, this function synchronizes that device with the CPU.

NOTE Note Consider using torch.linalg.lstsq() if possible for multiplying a matrix on the left by the pseudoinverse, as: torch.linalg.lstsq(A, B).solution == A.pinv() @ B It is always preferred to use lstsq() when possible, as it is faster and more numerically stable than computing the pseudoinverse explicitly.

NOTE Note This function has NumPy compatible variant linalg.pinv(A, rcond, hermitian=False). However, use of the positional argument rcond is deprecated in favor of rtol.

⚠ WARNING Warning This function uses internally torch.linalg.svd() (or torch.linalg.eigh() when hermitian= True), so its derivative has the same problems as those of these functions. See the warnings in torch.linalg.svd() and torch.linalg.eigh() for more details.

NOTE See also torch.linalg.inv() computes the inverse of a square matrix. torch.linalg.lstsq() computes A.pinv() @ B with a numerically stable algorithm.

Detailed Documentation

Documentation

Computes the pseudoinverse (Moore-Penrose inverse) of a matrix.

The pseudoinverse may be defined algebraically but it is more computationally convenient to understand it through the SVD

Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if A is a batch of matrices then the output has the same batch dimensions.

If hermitian= True, A is assumed to be Hermitian if complex or symmetric if real, but this is not checked internally. Instead, just the lower triangular part of the matrix is used in the computations.

The singular values (or the norm of the eigenvalues when hermitian= True) that are below max(atol,σ1·rtol)\max(\text{atol}, \sigma_1 \cdot \text{rtol})max(atol,σ1·rtol) threshold are treated as zero and discarded in the computation, where σ1\sigma_1σ1 is the largest singular value (or eigenvalue).

If rtol is not specified and A is a matrix of dimensions (m, n), the relative tolerance is set to be rtol=max(m,n)ε\text{rtol} = \max(m, n) \varepsilonrtol=max(m,n)ε and ε\varepsilonε is the epsilon value for the dtype of A (see finfo). If rtol is not specified and atol is specified to be larger than zero then rtol is set to zero.

If atol or rtol is a torch.Tensor, its shape must be broadcastable to that of the singular values of A as returned by torch.linalg.svd().

This function uses torch.linalg.svd() if hermitian= False and torch.linalg.eigh() if hermitian= True. For CUDA inputs, this function synchronizes that device with the CPU.

Consider using torch.linalg.lstsq() if possible for multiplying a matrix on the left by the pseudoinverse, as:

It is always preferred to use lstsq() when possible, as it is faster and more numerically stable than computing the pseudoinverse explicitly.

This function has NumPy compatible variant linalg.pinv(A, rcond, hermitian=False). However, use of the positional argument rcond is deprecated in favor of rtol.

This function uses internally torch.linalg.svd() (or torch.linalg.eigh() when hermitian= True), so its derivative has the same problems as those of these functions. See the warnings in torch.linalg.svd() and torch.linalg.eigh() for more details.

torch.linalg.inv() computes the inverse of a square matrix.

torch.linalg.lstsq() computes A.pinv() @ B with a numerically stable algorithm.

A (Tensor) – tensor of shape (*, m, n) where * is zero or more batch dimensions.

rcond (float, Tensor, optional) – [NumPy Compat]. Alias for rtol. Default: None.

atol (float, Tensor, optional) – the absolute tolerance value. When None it’s considered to be zero. Default: None.

rtol (float, Tensor, optional) – the relative tolerance value. See above for the value it takes when None. Default: None.

hermitian (bool, optional) – indicates whether A is Hermitian if complex or symmetric if real. Default: False.

out (Tensor, optional) – output tensor. Ignored if None. Default: None.
